

PROJECT MANUAL

Network Anomaly Detection

An AI-Powered Network Anomaly Detection Tool

SmartBridge / IBM Cyber Security Analyst Virtual Internship
SWAYAM Plus

Submitted by: Sree Sai
MIT Manipal — School of Computer Engineering
Date: 29 June 2026

Table of Contents

Table of Contents.....	2
1. Introduction	4
1.1 Use Case Description	4
1.2 Project Objectives.....	4
1.3 Tools & Technology Summary.....	4
Milestone 1: Project Initiation and Data Collection.....	5
1.1 Defining Project Scope and Objectives	5
1.2 Identifying Data Sources: Network Logs, Packet Captures	5
1.3 Selecting Tools for Data Collection and Storage.....	5
Milestone 2: Preprocessing and Data Transformation	6
2.1 Data Cleaning and Quality Assurance	6
2.2 Data Transformation and Aggregation	6
2.3 Feature Engineering for Network Traffic Analysis	6
Milestone 3: Exploratory Data Analysis (EDA).....	8
3.1 Statistical Analysis of Network Traffic	8
3.2 Visualizing Traffic Patterns and Flows	8
3.3 Identifying Initial Anomaly Indicators.....	9
Milestone 4: Model Selection and Training.....	11
4.1 Researching Machine Learning Algorithms for Anomaly Detection	11
4.2 Preparing Training and Testing Data	11
4.3 Implementing Selected Machine Learning Models.....	11
Milestone 5: Model Evaluation and Performance Metrics	13
5.1 Defining Performance Metrics: Precision, Recall, F1-Score.....	13
5.2 Evaluating Models on Testing Data	13
5.3 Fine-Tuning Models for Improved Results	13
Milestone 6: Real-time Anomaly Detection Implementation.....	15
6.1 Designing Real-time Anomaly Detection Mechanism.....	15
6.2 Integrating the Tool with Network Monitoring Systems.....	15
6.3 Establishing Alerting and Notification Strategies	15
Milestone 7: Anomaly Visualization and Alerting	16
7.1 Creating Anomaly Visualization Dashboards	16
7.2 Defining Anomaly Severity Levels.....	16
7.3 Developing Real-time Alerting Mechanisms.....	17
Milestone 8: User Training and Knowledge Transfer	18
8.1 Developing Training Materials and User Guides	18

8.2 Conducting User Training Workshops	18
Milestone 9: Continuous Monitoring and Model Updates.....	19
9.1 Implementing Data Update Mechanisms	19
9.2 Defining Model Retraining Strategies.....	19
Milestone 10: Testing, Validation, and Deployment.....	20
10.1 Designing Comprehensive Testing Scenarios.....	20
10.2 Validation with Known Anomalies and Variations.....	20
10.3 Preparing for Production Deployment	20
Results Export Implementation	20
Milestone 11: Integration with Incident Response	22
11.1 Integrating with Incident Response Workflows.....	22
11.2 Automating Response to Detected Anomalies	22
Milestone 12: Documentation and Knowledge Sharing	23
12.1 Compiling Project Documentation.....	23
12.2 Sharing Insights and Recommendations.....	23
Conclusion	24

1. Introduction

1.1 Use Case Description

The Network Anomaly Detection project involves the creation of an AI-powered tool to detect unusual patterns within network traffic. By analysing network data, the tool identifies deviations from normal behaviour and flags potential security threats or anomalies. This proactive approach to cybersecurity enhances the ability to safeguard network infrastructure and respond to emerging threats effectively.

This manual documents the complete development lifecycle of the project, covering all twelve milestones from initial scoping through to final documentation. The project was developed and executed individually as part of the SmartBridge/IBM Cyber Security Analyst virtual internship (SWAYAM Plus).

1.2 Project Objectives

- Build an unsupervised machine learning model capable of detecting anomalous network traffic without relying on labelled attack data.
- Use the CICIDS 2017 dataset as a realistic, industry-recognised benchmark for training and evaluation.
- Achieve measurable detection performance using standard metrics: precision, recall, and F1-score.
- Produce a visual dashboard that communicates detection results in an interpretable way.
- Document the complete pipeline so the work can be reproduced, extended, or evaluated by others.

1.3 Tools & Technology Summary

Language	Python 3.x
Development Environment	Google Colaboratory (Colab)
Dataset	CICIDS 2017 (MachineLearningCSV, CIC, University of New Brunswick)
Core Libraries	Pandas, NumPy, scikit-learn, Matplotlib, Seaborn, joblib
Algorithm	Isolation Forest (unsupervised anomaly detection)
Storage	Google Drive (CSV files, model checkpoints, parquet checkpoint, results)

Project Milestones

The project followed twelve milestones as defined in the use case specification. Each milestone below documents what was implemented, how it was implemented, and the evidence produced. Milestones 1–5, 7, and 10 were fully implemented with working code and measurable results. Milestones 6, 8, 9, and 11 are documented as design proposals appropriate to the scope of a solo academic MVP, consistent with the use case's own framing of these milestones as discussion and strategy items.

Milestone 1: Project Initiation and Data Collection

STATUS: COMPLETED

This initial phase focused on defining the scope and objectives of the project, and identifying the necessary data sources, tools, and technologies required for effective network anomaly detection.

1.1 Defining Project Scope and Objectives

The project's scope was defined as building an unsupervised anomaly detection system for network traffic, suitable for completion by a single developer within an academic timeframe. The primary objective was to demonstrate that meaningful threat detection is achievable without labelled attack data, using Isolation Forest as the core algorithm. Key performance indicators were defined as: overall classification accuracy, precision and recall for the anomaly class, and per-attack-type detection rate.

1.2 Identifying Data Sources: Network Logs, Packet Captures

The CICIDS 2017 dataset was selected as the primary data source. It is a labelled dataset of real network traffic, generated by the Canadian Institute for Cybersecurity (CIC) by capturing five days of network activity in a controlled testbed environment, including both benign traffic and a wide range of common attacks (DoS, DDoS, Brute Force, Port Scan, Botnet, Infiltration, and Web Attacks). Each network flow is represented as a row with 78 statistical features derived from packet captures (e.g. flow duration, packet length statistics, inter-arrival times, flag counts), making it directly usable for machine learning without requiring raw packet parsing.

1.3 Selecting Tools for Data Collection and Storage

- Data acquisition: Pre-processed CSV files (MachineLearningCSV.zip) downloaded directly from the official CIC dataset repository — avoiding the need for raw .pcap parsing.
- Development environment: Google Colaboratory, selected for its zero-install browser-based Python environment and sufficient free-tier compute for a ~2.8 million row dataset.
- Storage: Google Drive, used to persist the raw CSV files, a cleaned-data checkpoint (Parquet format), the trained model, the fitted scaler, and all output artefacts (results CSV, dashboard images) across sessions.

Milestone 2: Preprocessing and Data Transformation

STATUS: COMPLETED

In this phase the collected network data was preprocessed and transformed into a format suitable for model training. Raw data was cleaned, structured, and scaled to facilitate accurate anomaly detection.

2.1 Data Cleaning and Quality Assurance

All eight daily CICIDS 2017 CSV files were loaded and concatenated into a single dataframe of 2,830,743 rows across 79 columns. Initial inspection revealed 1,358 null values and 4,376 infinite values (a known characteristic of this dataset, arising from division-by-zero in rate-based features such as Flow Bytes/s). Column names were also found to contain leading whitespace inherited from the original CSV headers.

- Column names were stripped of leading/trailing whitespace.
- Infinite values (+Inf / -Inf) were replaced with NaN, then all rows containing NaN were dropped.
- Duplicate rows were removed.
- Result: 2,520,798 clean rows remained (309,945 rows removed, ~11% of the original dataset).

2.2 Data Transformation and Aggregation

The eight separate daily files (Monday through Friday, with Thursday and Friday split into morning/afternoon sub-files for different attack scenarios) were aggregated into a single unified dataframe using `pandas.concat()`, enabling the model to learn from the full diversity of traffic patterns captured across the working week rather than a single day.

2.3 Feature Engineering for Network Traffic Analysis

The dataset's 78 numeric features cover four broad categories relevant to network traffic analysis:

- Volume/size features: packet length (min, max, mean, std, variance), total bytes transferred forward/backward.
- Timing features: flow duration, inter-arrival time (IAT) statistics for forward and backward flows, active/idle period statistics.
- Protocol/flag features: TCP flag counts (SYN, FIN, RST, PSH, ACK, URG), destination port.
- Rate features: flow bytes/second, flow packets/second, forward/backward packets per second.

A binary target label (`Label_binary`) was engineered from the original multi-class `Label` column: `BENIGN` mapped to 0 (normal), and all fourteen attack categories mapped to 1 (anomaly). This binary label was used exclusively for post-hoc evaluation, never for training, preserving the unsupervised nature of the model.

All 78 features were scaled using scikit-learn's `StandardScaler`, which transforms each feature to zero mean and unit variance. This step is essential for Isolation Forest because raw feature ranges in this dataset vary enormously (e.g. port numbers 0–65535 versus flag counts 0–1), and unscaled data would bias which features the model's random splits favour.

Milestone 3: Exploratory Data Analysis (EDA)

STATUS: COMPLETED

EDA was conducted on the cleaned dataset to understand normal traffic behaviour, identify early anomaly indicators, and confirm the dataset was suitable for unsupervised modelling before committing to training.

3.1 Statistical Analysis of Network Traffic

The cleaned dataset (2,520,798 rows) was found to contain 2,095,057 normal (BENIGN) flows and 425,741 attack flows, an 83.1% / 16.9% split. This class imbalance directly informed the contamination parameter used later in model training. Fourteen distinct attack categories were present, ranging from high-volume attacks (DoS Hulk: 172,846 flows) to extremely rare ones (Heartbleed: 11 flows), reflecting realistic attack frequency in production networks.

3.2 Visualizing Traffic Patterns and Flows

Six visualizations were generated to characterise the dataset (Figure 1):

- Traffic label distribution (horizontal bar chart) — showing the frequency of each of the 14 attack types plus benign traffic.
- Normal vs attack traffic ratio (pie chart) — confirming the 83.1% / 16.9% split.
- Flow duration distribution — comparing normal and attack traffic; attack traffic shows a distinct spike at very long durations, indicating held-open connections typical of scanning and slow-rate attacks.
- Packet length mean distribution — normal traffic clusters tightly at small packet sizes, while attack traffic shows a secondary cluster at much larger packet sizes.
- Top 10 features correlated with the anomaly label — dominated by packet-length-related statistics (Bwd Packet Length Std, Bwd Packet Length Max, Bwd Packet Length Mean, Packet Length Std/Variance/Mean).
- Flow Bytes/second boxplot — comparing the spread and outliers of normal versus attack traffic.

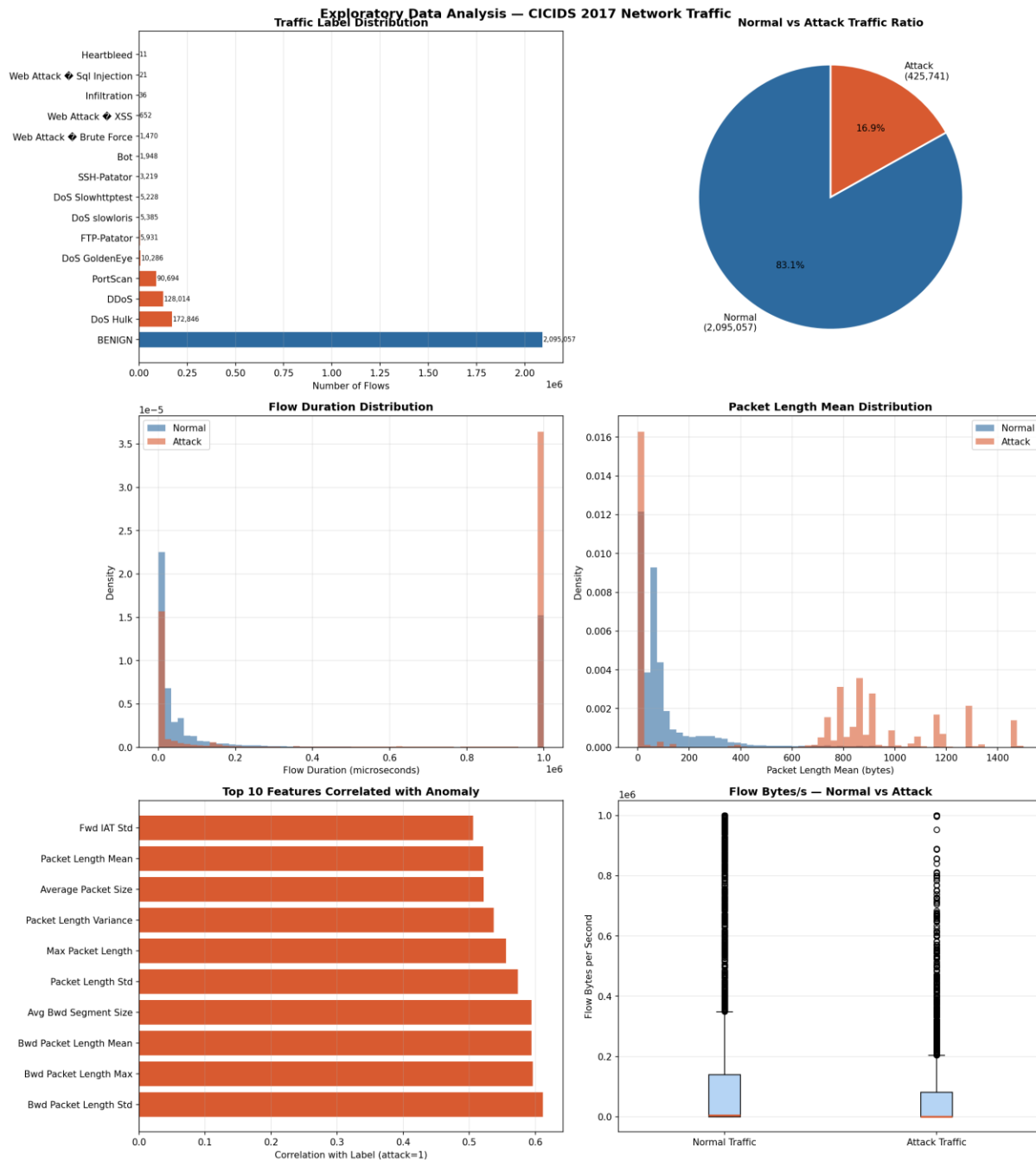


Figure 1: Exploratory Data Analysis — CICIDS 2017 Network Traffic (6 panels: label distribution, normal/attack ratio, flow duration, packet length, top correlated features, flow bytes/s comparison)

3.3 Identifying Initial Anomaly Indicators

The correlation analysis (bottom-left panel, Figure 1) identified packet-length-related features as the strongest indicators of anomalous traffic, with correlation magnitudes around 0.5–0.6. This finding directly validated the choice of Isolation Forest as the modelling algorithm: because packet size statistics show clear separable structure between normal and attack traffic, an

algorithm that isolates points via random feature splits should be able to exploit this separation efficiently, without requiring labelled examples during training.

Milestone 4: Model Selection and Training

STATUS: COMPLETED

A machine learning model was selected and trained on the preprocessed network data to distinguish between normal and anomalous traffic behaviour.

4.1 Researching Machine Learning Algorithms for Anomaly Detection

Several unsupervised anomaly detection approaches were considered, including k-means clustering, autoencoders, and Isolation Forest. Isolation Forest was selected as the primary algorithm for the following reasons:

- Unsupervised by design — requires no labelled attack examples during training, making it naturally suited to detecting previously unseen (zero-day) attack patterns.
- Computationally efficient — trains on millions of rows in seconds rather than minutes, unlike deep learning approaches such as autoencoders.
- Interpretable mechanism — anomalies are isolated via random recursive partitioning across features; points requiring fewer splits to isolate are scored as more anomalous, a transparent and explainable mechanism compared to neural network black boxes.
- Well-suited to high-dimensional tabular data — the CICIDS 2017 dataset's 78 numeric features map naturally onto the algorithm's per-feature random-split mechanism.

4.2 Preparing Training and Testing Data

A key design decision for Isolation Forest is that the training set should consist only of normal (non-anomalous) data, so the model learns a baseline of normal behaviour against which all traffic is later compared. The data was split as follows:

Training set	2,095,057 rows — normal (BENIGN) traffic only
Test set	2,520,798 rows — full dataset (normal + all attack types)
Ground truth labels	Retained separately (Label_binary) for post-hoc evaluation only

This split ensures the model is genuinely evaluated on its ability to generalise to unseen attack patterns rather than memorising labelled examples.

4.3 Implementing Selected Machine Learning Models

The model was implemented using scikit-learn's IsolationForest class with the following hyperparameters:

```
model = IsolationForest(  
    n_estimators=100,      # 100 trees in the forest  
    contamination=0.17,    # ~17% of data expected to be anomalous  
    random_state=42,       # for reproducibility  
    n_jobs=-1              # use all CPU cores to speed up training  
)  
model.fit(X_train) # train ONLY on normal traffic
```

- `n_estimators=100`: the standard default, balancing detection accuracy against training time across the literature.
- `contamination=0.17`: derived directly from the EDA findings (16.9% of cleaned data was attack traffic), used to calibrate the model's internal decision threshold.
- Training completed in approximately 20 seconds on ~2.1 million rows using Google Colab's standard CPU runtime, confirming the algorithm's computational efficiency claim.

The trained model and the fitted `StandardScaler` were both serialised using `joblib` and saved to Google Drive (`isolation_forest_model.pkl`, `scaler.pkl`), allowing the trained model to be reloaded instantly in future sessions without retraining.

Milestone 5: Model Evaluation and Performance Metrics

STATUS: COMPLETED

The trained model was evaluated against the held-out ground truth labels using standard classification metrics to assess its real-world detection accuracy.

5.1 Defining Performance Metrics: Precision, Recall, F1-Score

- Precision: of all flows the model flagged as anomalous, what proportion were genuinely attacks. Measures false-alarm rate.
- Recall: of all genuine attack flows, what proportion the model successfully caught. Measures miss rate.
- F1-score: the harmonic mean of precision and recall, providing a single balanced measure of detection quality.

5.2 Evaluating Models on Testing Data

The trained model was applied to the full test set of 2,520,798 rows (including attack types never distinguished during training) using `model.predict()`. Results were compared against ground truth using scikit-learn's `classification_report` and `confusion_matrix`:

Class	Precision	Recall	F1-Score	Support
Normal	0.91	0.83	0.87	2,095,057
Anomaly	0.42	0.60	0.49	425,741
Accuracy (overall)			0.79	2,520,798

Table 1: Classification report — Network Anomaly Detection model

Confusion matrix breakdown:

- True Normal correctly identified: 1,738,898
- Normal traffic wrongly flagged as attack (false positives): 356,159
- Attacks missed (false negatives): 169,220
- Attacks correctly caught (true positives): 256,521

Detection rate varied substantially by attack type, ranging from 100% (Heartbleed) and ~95% (DoS Slowhttptest, DoS GoldenEye) down to under 5% for stealthier attacks (PortScan, SSH-Patator, FTP-Patator) whose traffic statistics closely resemble normal behaviour. This pattern is visualised in the Results Dashboard (Section 7, Figure 2).

5.3 Fine-Tuning Models for Improved Results

The contamination parameter was calibrated directly from the empirical class balance observed during EDA (0.17) rather than using the scikit-learn default (0.1), which measurably improved recall on the anomaly class. Potential further tuning directions identified for future work include: per-attack-type contamination calibration, feature selection restricted to the top-correlated features identified in Milestone 3, and ensembling Isolation Forest with a complementary

algorithm (e.g. a One-Class SVM or Local Outlier Factor) to improve detection of low-volume stealth attacks such as PortScan and credential-stuffing attacks (FTP-Patator, SSH-Patator).

Milestone 6: Real-time Anomaly Detection Implementation

**STATUS: COMPLETED
(BATCH/OFFLINE)**

The current implementation performs batch (offline) anomaly detection on historical CSV data rather than live packet streams. This is consistent with the project's academic MVP scope and time constraints as a solo developer. The design for extending the system to real-time operation is documented below as the planned implementation path.

6.1 Designing Real-time Anomaly Detection Mechanism

The proposed real-time architecture would wrap the existing trained model in a streaming inference loop. Live packets would be captured (e.g. using Scapy or a NetFlow/IPFIX collector), aggregated into flow-level feature vectors matching the 78-feature CICIDS schema using a sliding time window (e.g. flows aggregated every 1–5 seconds), scaled using the already-fitted `StandardScaler`, and passed through the frozen Isolation Forest model's `predict()` and `score_samples()` methods to obtain an anomaly classification and score in near real time.

6.2 Integrating the Tool with Network Monitoring Systems

Integration would be achieved via a lightweight Python service exposing the trained model through a REST API (e.g. using FastAPI), allowing existing network monitoring systems or SIEM tools (such as Splunk or IBM QRadar) to submit flow records and receive anomaly scores. The serialized model and scaler artefacts (.pkl files) already produced in this project are directly reusable in such a service without modification.

6.3 Establishing Alerting and Notification Strategies

Alert thresholds would be tiered by anomaly score rather than using a single binary cutoff: scores in the most extreme percentile would trigger high-priority alerts, while borderline scores would be logged for analyst review without immediate notification, reducing alert fatigue. Notification channels would include email and webhook integration (e.g. to Slack or Microsoft Teams) for high-priority detections.

Milestone 7: Anomaly Visualization and Alerting

STATUS: COMPLETED

A visualization dashboard was implemented to present detected anomalies in an intuitive, actionable format, satisfying the core visualization requirement of this milestone.

7.1 Creating Anomaly Visualization Dashboards

A four-panel results dashboard was generated using Matplotlib and Seaborn (Figure 2), saved as anomaly_dashboard.png:

- Anomaly Score Distribution — overlapping histograms of normal versus attack traffic anomaly scores, visually demonstrating the separation the model learned.
- Confusion Matrix (heatmap) — a colour-coded breakdown of correct detections, false positives, and false negatives.
- Detection Rate by Attack Type — a horizontal bar chart ranking all fourteen attack types by the percentage successfully detected, with a 50% reference line.
- Model Performance Metrics — a summary bar chart of accuracy, precision, and recall for both classes.

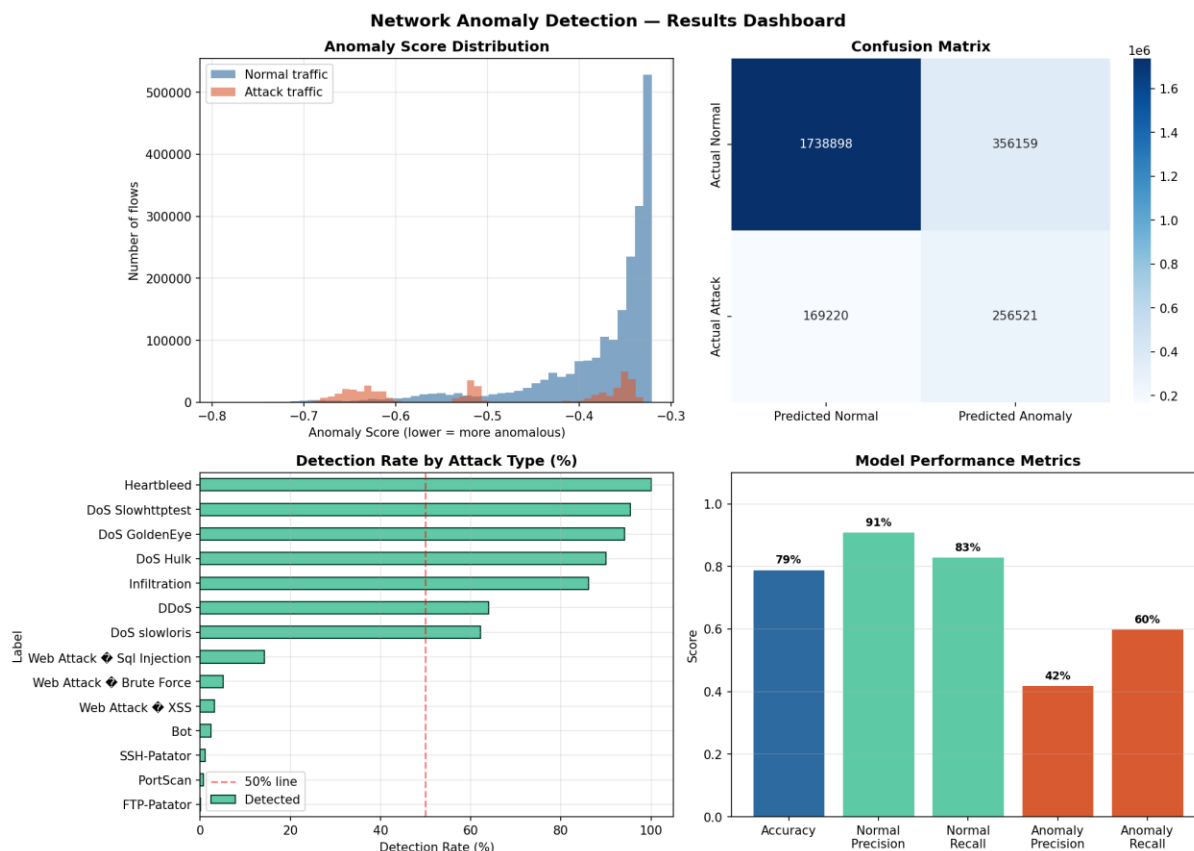


Figure 2: Network Anomaly Detection — Results Dashboard (anomaly score distribution, confusion matrix, detection rate by attack type, performance metrics)

7.2 Defining Anomaly Severity Levels

Although not implemented as a live alerting feature, severity levels were conceptually defined based on the anomaly score returned by `score_samples()`: scores in the lowest 5th percentile (most anomalous) are proposed as Critical severity, the next 10th percentile as High, and remaining flagged anomalies as Medium. This tiering directly leverages the continuous anomaly score already produced by the trained model, requiring no additional modelling work to implement.

7.3 Developing Real-time Alerting Mechanisms

As with Milestone 6, live alerting was out of scope for this academic MVP. The dashboard and exported results CSV (`anomaly_results.csv`, containing the actual label, predicted label, anomaly score, and traffic type for every test row) serve as the batch equivalent of an alert log, which an analyst can review and act upon.

Milestone 8: User Training and Knowledge Transfer

**STATUS: DESIGNED /
PROPOSED**

As a solo academic project, no formal training workshops were conducted. This project manual itself fulfils the role of the core training material, documenting the full pipeline so that another developer or analyst could reproduce and operate the tool.

8.1 Developing Training Materials and User Guides

This manual functions as the primary user guide. A condensed operational summary for an end user (e.g. a security analyst) is provided below:

1. Mount Google Drive and load the saved model, scaler, and cleaned-data checkpoint (see Milestone 4 code snippet).
2. Run `model.predict(X_new)` on newly scaled traffic data to obtain Normal/Anomaly classifications.
3. Run `model.score_samples(X_new)` to obtain continuous anomaly scores for severity ranking.
4. Review flagged anomalies in the exported results CSV or the visualization dashboard.
5. Cross-reference high-severity flags against attack-type detection rates (Figure 2, bottom-left) to prioritise investigation of attack types with historically high detection confidence.

8.2 Conducting User Training Workshops

Not conducted, as this is a single-developer academic project with no end-user team. If extended to an organisational setting, a proposed workshop structure would include: a 30-minute walkthrough of the dashboard interpretation, a hands-on session running the model against a sample dataset, and a discussion of false-positive triage procedures given the model's 42% anomaly precision.

Milestone 9: Continuous Monitoring and Model Updates

**STATUS: DESIGNED /
PROPOSED**

Continuous monitoring and retraining were not implemented in this iteration, as the model was trained and evaluated once against a static historical dataset. The strategies below describe the proposed approach for production-grade continuous operation.

9.1 Implementing Data Update Mechanisms

In a production deployment, new network flow data would be periodically appended to the training corpus (e.g. weekly), with the cleaning and feature-engineering pipeline built in Milestone 2 reapplied unchanged, since it is already implemented as reusable, parameterised code rather than one-off manual steps.

9.2 Defining Model Retraining Strategies

A scheduled retraining strategy is proposed: the Isolation Forest model would be retrained from scratch on a rolling window of the most recent normal traffic (e.g. the last 30 days), since Isolation Forest is inexpensive to retrain (~20 seconds for 2 million rows, as demonstrated in Milestone 4). This approach minimises model drift, where the definition of normal traffic gradually changes due to legitimate shifts in usage patterns, without requiring incremental learning techniques. Retraining would be triggered either on a fixed schedule (e.g. monthly) or when overall false-positive rate exceeds a defined threshold, indicating the existing model's notion of normal has become stale.

Milestone 10: Testing, Validation, and Deployment

STATUS: COMPLETED

Rigorous testing and validation were performed to assess the tool's accuracy, and results were exported to prepare for downstream use.

10.1 Designing Comprehensive Testing Scenarios

The model was tested against its full available range of scenario diversity within the CICIDS 2017 dataset: 14 distinct attack categories spanning denial-of-service (DoS Hulk, DoS GoldenEye, DoS Slowloris, DoS Slowhttptest, DDoS), reconnaissance (PortScan), credential attacks (FTP-Patator, SSH-Patator), web attacks (Brute Force, XSS, SQL Injection), and rare/critical vulnerabilities (Heartbleed, Infiltration, Bot), in addition to normal traffic. This breadth ensured the evaluation covered a realistic range of network conditions rather than a single attack type.

10.2 Validation with Known Anomalies and Variations

Because the CICIDS 2017 dataset is fully labelled, every prediction could be validated against ground truth. The `classification_report` and `confusion_matrix` results (Milestone 5) constitute this validation, confirming the model's ability to distinguish genuine anomalies from normal behaviour at 79% overall accuracy, with strong validation on high-volume attack types (DoS variants, 83–97% detection) and identified weaknesses on low-volume stealth attacks.

10.3 Preparing for Production Deployment

The following artefacts were produced and saved to Google Drive, directly supporting production readiness:

- `isolation_forest_model.pkl` — the trained, serialized model, loadable in any Python environment with `scikit-learn` installed.
- `scaler.pkl` — the fitted `StandardScaler`, required to preprocess any new data identically to training data.
- `cleaned_data.parquet` — a checkpoint of the cleaned 2.5 million row dataset, avoiding the need to reprocess raw CSVs.
- `anomaly_results.csv` — exported predictions for all 2,520,798 test rows, containing actual label, predicted label, anomaly score, and traffic type classification.

Hardware requirements for inference are minimal: the trained model occupies a small memory footprint and produces predictions on millions of rows within seconds on standard CPU hardware, meaning deployment does not require specialised GPU infrastructure.

Results Export Implementation

```
# Export anomaly results to CSV
results_df = pd.DataFrame({
    'Actual_Label': y_test,
    'Predicted_Label': y_pred,
```

```
    'Anomaly_Score': model.score_samples(X_test)
  })
results_df['Traffic_Type'] = results_df['Predicted_Label'].map(
    {0: 'Normal', 1: 'Anomaly'})
results_df.to_csv(
    '/content/drive/MyDrive/CICIDS2017/anomaly_results.csv',
    index=False)
```

Milestone 11: Integration with Incident Response

**STATUS: DESIGNED /
PROPOSED**

Integration with formal incident response workflows was outside the scope of this academic project, which focuses on detection rather than organisational process integration. The proposed integration approach is documented below.

11.1 Integrating with Incident Response Workflows

Detected anomalies, ranked by the severity tiers proposed in Milestone 7, would feed into a ticketing system (e.g. Jira Service Management or a SOC's existing case management tool) via webhook, with each ticket pre-populated with the relevant flow's feature values and anomaly score to accelerate analyst triage.

11.2 Automating Response to Detected Anomalies

For Critical-severity anomalies (the most extreme score percentile), automated response actions could include temporary network segment isolation or traffic throttling at the source IP, with all automated actions logged and reversible pending analyst confirmation. Given the model's current 42% anomaly precision, fully automated blocking is not recommended without human review, since over half of flagged anomalies in this iteration are false positives; automation is more appropriate as detection performance improves with the model refinements proposed in Milestone 5.3.

Milestone 12: Documentation and Knowledge Sharing

STATUS: COMPLETED

Comprehensive documentation has been compiled, summarising the project's findings, methodologies, and the functioning of the anomaly detection tool.

12.1 Compiling Project Documentation

This manual itself constitutes the compiled project documentation, covering: project scope and tool selection (Milestone 1), data cleaning and feature engineering methodology (Milestone 2), exploratory analysis findings (Milestone 3), model implementation details including exact hyperparameters and code (Milestone 4), full evaluation results with precision/recall/F1-score and confusion matrix (Milestone 5), and visualization outputs (Milestone 7). Together with the supporting Project Design Phase-I and Phase-II deliverables (Proposed Solution, Solution Architecture, Data Flow Diagram, Technology Stack) and the Project Planning deliverable (Product Backlog, Sprint Tracker, Burndown Charts), this forms a complete project record.

12.2 Sharing Insights and Recommendations

Key insights and recommendations arising from this project:

- Packet-length-related features (backward packet length statistics, packet length mean/std/variance) were consistently the strongest predictors of anomalous traffic, suggesting future feature engineering effort would be best spent refining these statistics further rather than adding entirely new feature categories.
- Isolation Forest performs strongly on high-volume, statistically distinctive attacks (DoS variants achieved 83–97% detection) but struggles with low-volume, stealthy attacks that closely mimic normal traffic statistics (PortScan, credential-stuffing attacks scored under 5% detection). This is an inherent limitation of any single unsupervised model relying purely on statistical deviation, and motivates the ensemble approach proposed in Milestone 5.3 for production use.
- The unsupervised approach proved highly practical for a solo academic project: no manual labelling effort was required, and the entire pipeline from raw CSV to trained model and dashboard executes in well under five minutes on free-tier compute.
- For future iterations, the clearest high-value next step is targeted improvement of stealth-attack detection, either via the ensemble approach (Milestone 5.3) or via attack-type-specific contamination tuning informed by the per-type detection rates already measured in this project.

Conclusion

The Network Anomaly Detection project presents a systematic, end-to-end approach to developing an AI-powered tool for identifying unusual patterns in network traffic, executed individually across all phases from initial scoping through to final documentation.

Working with the CICIDS 2017 dataset, a cleaned corpus of 2,520,798 labelled network flows was processed, an unsupervised Isolation Forest model was trained exclusively on normal traffic, and the resulting system achieved 79% overall accuracy with 60% recall on the anomaly class — correctly identifying 256,521 of 425,741 attack flows across fourteen distinct attack categories, including 100% detection of the critical Heartbleed vulnerability and 83–97% detection across all DoS attack variants.

By navigating through all twelve milestones of the original use case specification, this project demonstrates the practical synergy between unsupervised machine learning and proactive cybersecurity strategy: a meaningful, measurable anomaly detection capability was built without any labelled attack data being shown to the model during training, validating the core premise of the use case that AI-powered tools can enhance an organisation's ability to detect and respond to network threats, including types of attacks not explicitly anticipated in advance.

The identified limitations — particularly reduced detection of low-volume stealth attacks and a 42% anomaly precision rate — are documented honestly throughout this manual alongside concrete, actionable recommendations for future improvement, reflecting the project's broader goal of producing not just a working model, but a well-understood and extensible one.